1. Introducción y Arquitectura General

ConciliaEx es una Single Page Application (SPA) construida sobre una arquitectura orientada a componentes con consumo de APIs en tiempo real. El sistema separa claramente la capa de presentación (React) de la capa de datos y autenticación (Supabase), garantizando escalabilidad, mantenibilidad y un desarrollo ágil.

2. Stack Tecnológico

2.1 Frontend — Capa de Presentación

Tecnología	Versión	Rol en el proyecto
React	^18	Librería de UI con renderizado declarativo basado en componentes reutilizables.
TypeScript	^5	Tipado estático que reduce errores en tiempo de desarrollo.
Tailwind CSS	^3	Framework utility-first para estilos en línea; garantiza responsive y bundle mínimo.
React Router	^6	Client-Side Routing; navegación sin recarga de página.
Zustand	^4	Gestión de estado global ligera y sin boilerplate (usuario, tema, favoritos).
Vite	^5	Bundler de nueva generación con HMR instantáneo.

2.2 Backend y Base de Datos

Servicio	Tecnología	Descripción
BaaS	Supabase	Plataforma open source equivalente a Firebase, con PostgreSQL como base de datos.
Base de datos	PostgreSQL	Sistema relacional con soporte para RLS, tipos avanzados y consultas complejas.

Autenticación	Supabase Auth	Gestión de JWT, registro, login email/password y OAuth2 (Google).
API Client	@supabase/supabase-js	SDK oficial tipado para interactuar con la base de datos y la autenticación.

3. Diseño de la Base de Datos

Las tablas están gestionadas con Row Level Security (RLS) de PostgreSQL para evitar accesos indebidos a través de la API de PostgREST que expone Supabase automáticamente.

3.1 Esquema Relacional

Tabla	Campos principales	Descripción
profiles	id_usuario (UUID, PK), id_perfil (FK → auth.users), rol (familia | entidad | admin), created_at	Vinculada a la tabla genérica de Supabase Auth. Almacena el rol de cada usuario registrado.
familias	id_familia (UUID, PK), id_perfil (FK → profiles), nombre_tutor, telefono, localidad, created_at	Datos específicos del perfil de familia. Cada familia tiene exactamente un registro en esta tabla.
entidades	id_entidad (UUID, PK), id_perfil (FK → profiles), cif, nombre_centro, direccion, verificada (bool), created_at	Registro de entidades sociales. El campo verificada indica si el administrador ha aprobado su publicación.
servicios	id_servicio (UUID, PK), id_entidad (FK), titulo, descripcion, tipo_servicio, localidad, activo (bool), created_at	Servicios publicados por cada entidad. Permite búsqueda y filtrado por tipo, localidad y estado activo.
favoritos	id_favorito (UUID, PK), id_familia (FK), id_servicio (FK), created_at	Tabla relacional N:M que guarda qué servicios ha marcado como favoritos cada familia.

4. Requisitos Funcionales

Los requisitos funcionales describen el comportamiento esperado del sistema desde el punto de vista del usuario.

ID	Nombre	Descripción

RF-01	Autenticación	El sistema permitirá registro e inicio de sesión mediante correo/contraseña y Google OAuth2.
RF-02	Roles	Existirán tres roles: familia, entidad y administrador, con accesos y vistas diferenciadas.
RF-03	Perfil de familia	Las familias podrán crear perfiles detallados de sus familiares con discapacidad.
RF-04	Catálogo de servicios	El sistema mostrará una lista de servicios filtrable por tipo, localidad y edad.
RF-05	Favoritos	Los usuarios familia podrán guardar y gestionar una lista de servicios favoritos.
RF-06	Gestión de entidades	Las entidades podrán crear, editar y eliminar sus servicios publicados.
RF-07	Panel de administración	El administrador gestionará usuarios, validará entidades y moderará contenido.
RF-08	Recuperación de contraseña	El sistema enviará un enlace seguro al correo para restablecer la contraseña.
RF-09	Notificaciones	El sistema notificará al usuario sobre nuevos servicios relevantes según su perfil.
RF-10	Buscador global	La plataforma incluirá un buscador con sugerencias automáticas en tiempo real.

5. Requisitos No Funcionales

ID	Categoría	Descripción
RNF-01	Accesibilidad	La interfaz cumplirá las pautas WCAG 2.1 nivel AA para máxima accesibilidad.
RNF-02	Rendimiento	Las búsquedas responderán en menos de 3 segundos bajo carga normal.

RNF-03	Seguridad	Contraseñas con bcrypt, comunicación HTTPS y cumplimiento RGPD.
RNF-04	Fiabilidad	Disponibilidad mínima del 95% garantizada por la infraestructura Supabase/Vercel.
RNF-05	Responsive	Diseño adaptado a móvil, tablet y escritorio mediante Tailwind CSS breakpoints.
RNF-06	Mantenibilidad	Código modular, tipado con TypeScript y documentado para facilitar ampliaciones.
RNF-07	Escalabilidad	Arquitectura basada en BaaS que permite escalar sin modificar la base de código.
RNF-08	Compatibilidad	Compatible con Chrome, Firefox, Edge y Safari en sus versiones actuales.

6. Seguridad y Políticas RLS

La seguridad del sistema se implementa en tres capas:

6.1 Autenticación (Supabase Auth)

- Contraseñas cifradas mediante bcrypt gestionado por Supabase.
- Tokens JWT con expiración configurable.
- Soporte para OAuth2 con Google (inicio de sesión sin contraseña).
- Recuperación de contraseña mediante enlace seguro por correo electrónico.

6.2 Autorización (RLS — Row Level Security)

Cada tabla tiene políticas RLS activas. Por ejemplo, la política sobre la tabla Familias es:

UPDATE solo si auth.uid() = id_perfil del registro

Esto garantiza que un usuario de rol familia solo pueda modificar su propio perfil y nunca acceder a datos de otras familias.

6.3 Comunicación Segura

- Toda la comunicación entre el cliente y Supabase se realiza sobre HTTPS/TLS.
- Las claves privadas del proyecto nunca se exponen al cliente; solo la Anon Key pública.

- Las variables de entorno sensibles se gestionan con archivos .env no versionados.